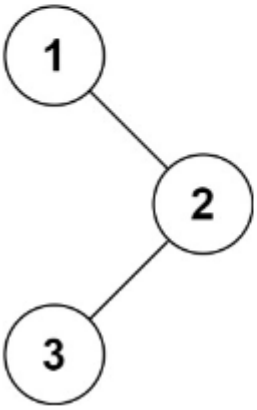
- **二叉树的中序遍历**
  - 链接：二叉树的中序遍历
  - 描述

94. 二叉树的中序遍历

简单 相关标签 相关企业 A文

给定一个二叉树的根节点 `root` ，返回 *它的* ***中序*** *遍历*。

示例 1：



> **输入：** `root = [1,null,2,3]`
> **输出：** `[1,3,2]`

示例 2：

> **输入：** `root = []`
> **输出：** `[]`

示例 3：

> **输入：** `root = [1]`
> **输出：** `[1]`

提示：

- 树中节点数目在范围 `[0, 100]` 内
- `-100 <= Node.val <= 100`

  - 代码

```
class TreeNode {
  val: number
  left: TreeNode | null
```

```
  right: TreeNode | null
  constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
    this.val = val === undefined ? 0 : val
    this.left = left === undefined ? null : left
    this.right = right === undefined ? null : right
  }
}

function inorderTraversal(root: TreeNode | null): number[] {
  const result: number[] = []
  const recursion = (treeNode: TreeNode) => {
    if (treeNode.left) {
      recursion(treeNode.left)
    }
    result.push(treeNode.val)
    if (treeNode.right) {
      recursion(treeNode.right)
    }
  }
  root && recursion(root)
  return result
}
```

- **二叉树的最大深度**
  - 链接：二叉树的最大深度
  - 描述

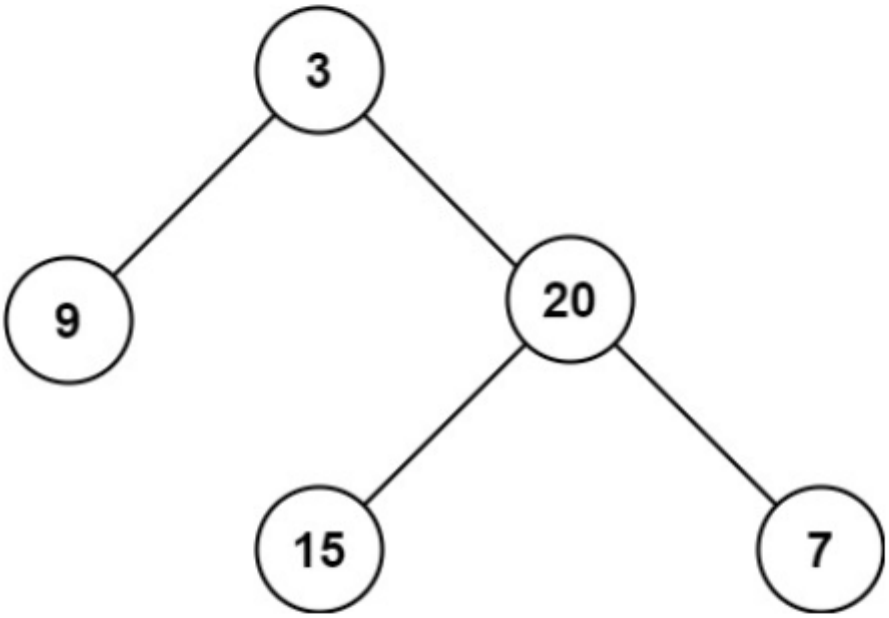
104. 二叉树的最大深度

简单 相关标签 相关企业 A文

给定一个二叉树 `root` ，返回其最大深度。

二叉树的 **最大深度** 是指从根节点到最远叶子节点的最长路径上的节点数。

示例 1：



> 输入：root = [3,9,20,null,null,15,7]
> 输出：3

示例 2：

> 输入：root = [1,null,2]
> 输出：2

提示：

- 树中节点的数量在 `[0, 10⁴]` 区间内。
- `-100 <= Node.val <= 100`
  - 答案

```
class TreeNode {
```

```
  val: number
  left: TreeNode | null
  right: TreeNode | null
  constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
    this.val = val === undefined ? 0 : val
    this.left = left === undefined ? null : left
    this.right = right === undefined ? null : right
  }
}

function maxDepth(root: TreeNode | null): number {
  if (root) {
    return Math.max(maxDepth(root.left), maxDepth(root.right)) + 1
  } else {
    return 0
  }
}
```

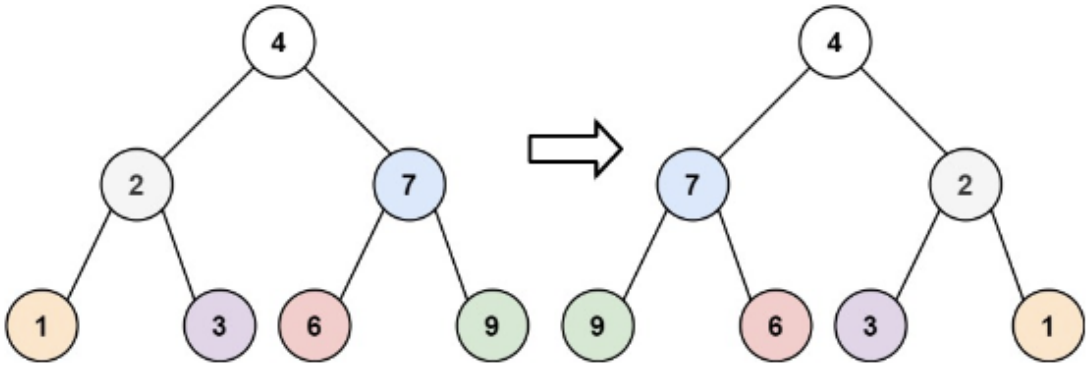
- **翻转二叉树**
  - 链接：翻转二叉树
  - 描述

226. 翻转二叉树

简单 相关标签 相关企业 Aa

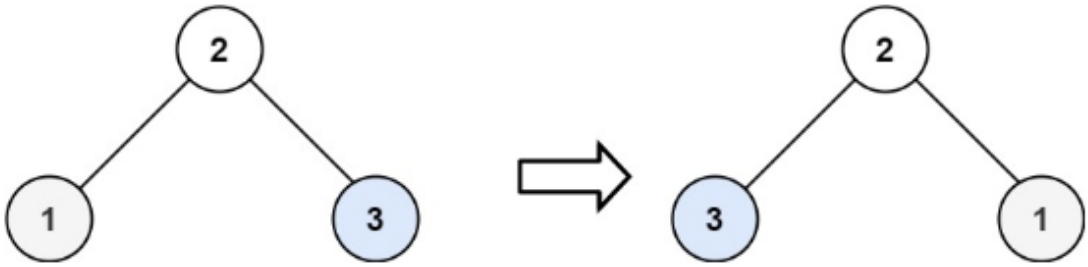
给你一棵二叉树的根节点 `root` ，翻转这棵二叉树，并返回其根节点。

示例 1：



> **输入：** `root = [4,2,7,1,3,6,9]`
> **输出：** `[4,7,2,9,6,3,1]`

示例 2：



> **输入：** `root = [2,1,3]`
> **输出：** `[2,3,1]`

示例 3：

> **输入：** `root = []`
> **输出：** `[]`

提示：

- 树中节点数目范围在 `[0, 100]` 内
- `-100 <= Node.val <= 100`

  - 答案

```
class TreeNode {
  val: number
  left: TreeNode | null
  right: TreeNode | null
  constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
    this.val = val === undefined ? 0 : val
    this.left = left === undefined ? null : left
    this.right = right === undefined ? null : right
  }
}

function invertTree(root: TreeNode | null): TreeNode | null {
  if (root) {
    ;[root.left, root.right] = [root.right, root.left]
    invertTree(root.left)
    invertTree(root.right)
  }
  return root
}
```

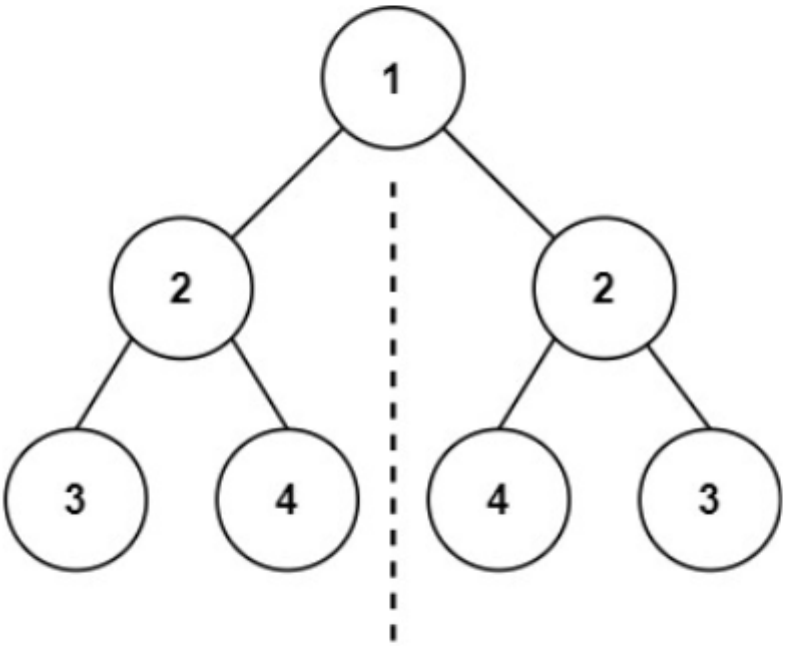
- **对称二叉树**
  - 链接：对称二叉树
  - 描述

101. 对称二叉树

简单 相关标签 相关企业 A文

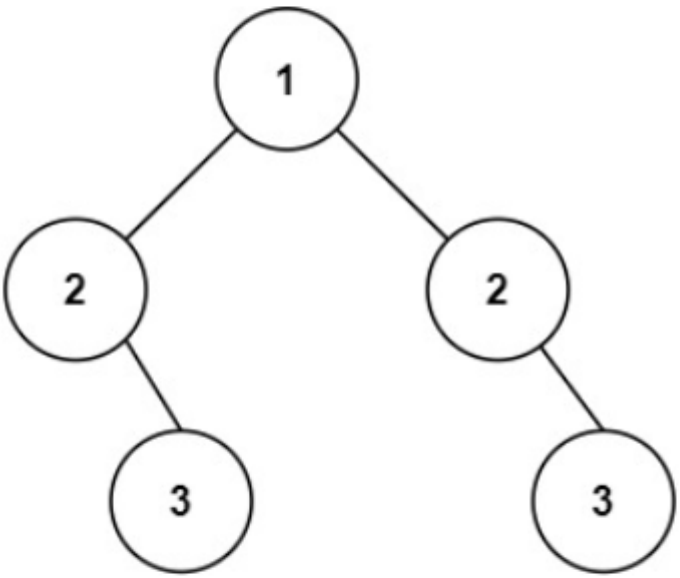
给你一个二叉树的根节点 `root` ， 检查它是否轴对称。

示例 1：



> **输入：** `root = [1,2,2,3,4,4,3]`
> **输出：** `true`

示例 2：



> **输入：** `root = [1,2,2,null,3,null,3]`
> **输出：** `false`

- 答案

提示：

```
class TreeNode {
  val: number
  left: TreeNode | null
  right: TreeNode | null
  constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
    this.val = val === undefined ? 0 : val
    this.left = left === undefined ? null : left
    this.right = right === undefined ? null : right
  }
}

function isSymmetric(root: TreeNode | null): boolean {
  return root ? checkMirror(root.left, root.right) : true
}

function checkMirror(node1: TreeNode | null, node2: TreeNode | null): boolean {
  if (node1 === null && node2 === null) return true
  if (node1 === null || node2 === null) return false
  return (
    node1.val === node2.val &&
    checkMirror(node1.left, node2.right) &&
    checkMirror(node1.right, node2.left)
  )
}
```

- **二叉树的直径**
  - 链接：二叉树的直径
  - 描述

543. 二叉树的直径

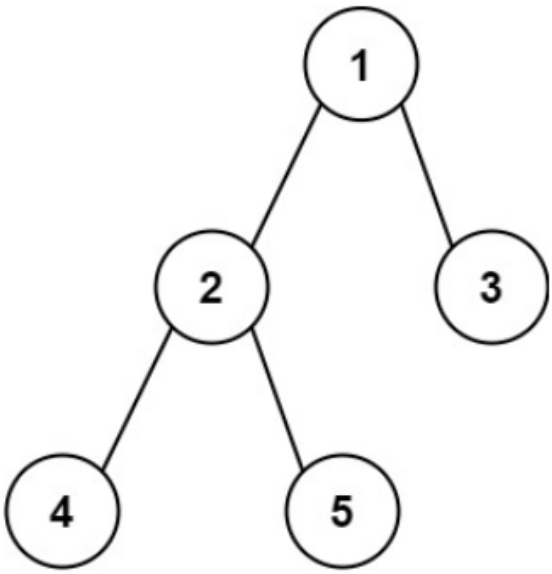
简单 相关标签 相关企业 A文

给你一棵二叉树的根节点，返回该树的 **直径** 。

二叉树的 **直径** 是指树中任意两个节点之间最长路径的 **长度** 。这条路径可能经过也可能不经过根节点 `root` 。

两节点之间路径的 **长度** 由它们之间边数表示。

示例 1：



> **输入：** root = [1,2,3,4,5]
> **输出：** 3
> **解释：** 3 , 取路径 [4,2,1,3] 或 [5,2,1,3] 的长度。

示例 2：

> **输入：** root = [1,2]
> **输出：** 1

提示：

- 树中节点数目在范围 $[1, 10^4]$ 内
- `-100 <= Node.val <= 100`

  - 答案

```
class TreeNode {
  val: number
  left: TreeNode | null
  right: TreeNode | null
```

```
  constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
    this.val = val === undefined ? 0 : val
    this.left = left === undefined ? null : left
    this.right = right === undefined ? null : right
  }
}

function diameterOfBinaryTree(root: TreeNode | null): number {
  let maxNode = 0
  recursion(root)

  function recursion(node: TreeNode | null): number {
    if (node === null) {
      return 0
    } else {
      const leftMax = recursion(node.left)
      const rightMax = recursion(node.right)
      maxNode = Math.max(maxNode, leftMax + rightMax + 1)
      return Math.max(leftMax, rightMax) + 1
    }
  }

  return maxNode - 1
}
```

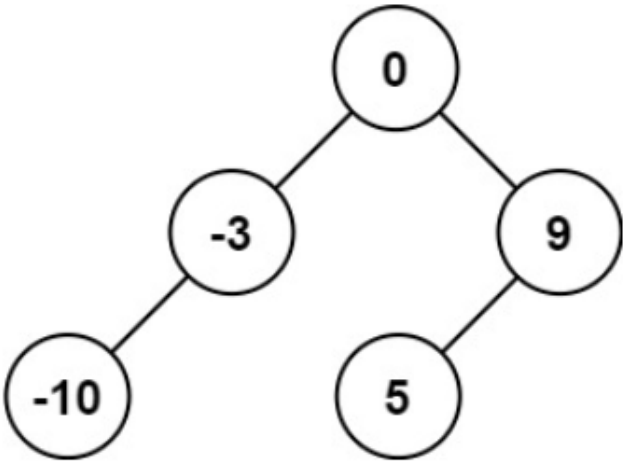
- **将有序数组转换为二叉搜索树**
  - 链接：将有序数组转换为二叉搜索树
  - 描述

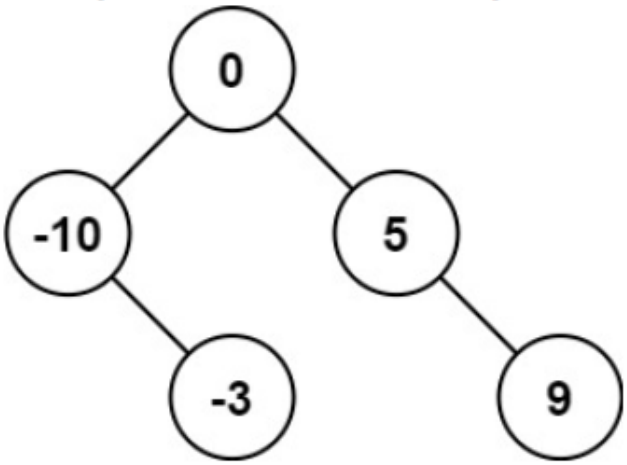
108. 将有序数组转换为二叉搜索树

简单 相关标签 相关企业 A文

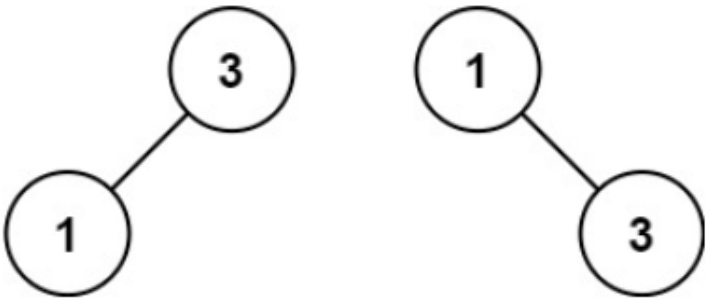
给你一个整数数组 `nums` ，其中元素已经按 **升序** 排列，请你将其转换为一棵 平衡 二叉搜索树。

示例 1：



> **输入：** nums = [-10,-3,0,5,9]
> **输出：** [0,-3,9,-10,null,5]
> **解释：** [0,-10,5,null,-3,null,9] 也将被视为正确答案：
>
> 
>

示例 2：



> **输入：** nums = [1,3]
> **输出：** [3,1]
> **解释：** [1,null,3] 和 [3,1] 都是高度平衡二叉搜索树。

- 答案

提示：

```
class TreeNode {
  val: number
  left: TreeNode | null
  right: TreeNode | null
  constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
    this.val = val === undefined ? 0 : val
    this.left = left === undefined ? null : left
    this.right = right === undefined ? null : right
  }
}

function sortedArrayToBST(nums: number[]): TreeNode | null {
  if (nums.length) {
    const midIndex = nums.length >> 1
    const node = new TreeNode(
      nums[midIndex],
      sortedArrayToBST(nums.slice(0, midIndex)),
      sortedArrayToBST(nums.slice(midIndex + 1))
    )
    return node
  } else {
    return null
  }
}
```

- 二叉树的层序遍历
  - 链接：二叉树的层序遍历
  - 描述

iShot (Purchase unlock to remove watermark)

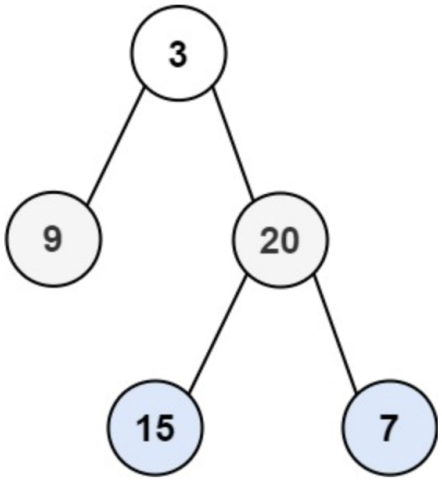
102. 二叉树的层序遍历

已解答

中等 相关标签 相关企业 提示

给你二叉树的根节点 `root` ，返回其节点值的 **层序遍历** 。 （即逐层地，从左到右访问所有节点）。

示例 1：



> **输入：** `root = [3,9,20,null,null,15,7]`
> **输出：** `[[3],[9,20],[15,7]]`

示例 2：

> **输入：** `root = [1]`
> **输出：** `[[1]]`

示例 3：

> **输入：** `root = []`
> **输出：** `[]`

提示：

- 树中节点数目在范围 `[0, 2000]` 内
- `-1000 <= Node.val <= 1000`

- 答案

```
class TreeNode {
  val: number
  left: TreeNode | null
  right: TreeNode | null
  constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
    this.val = val === undefined ? 0 : val
    this.left = left === undefined ? null : left
    this.right = right === undefined ? null : right
  }
}
```

```

function levelOrder(root: TreeNode | null): number[][] {
  const result: number[][] = []

  const loop = (treeNode: TreeNode | null, depth: number) => {
    if (treeNode) {
      if (!result[depth]) {
        result[depth] = []
      }
      result[depth].push(treeNode.val) // 拓展：如果想S形层级遍历，这里更改为 result[depth][depth % 2 === 0 ? 'push' : 'unshift'](treeNode.val)
      loop(treeNode.left, depth + 1)
      loop(treeNode.right, depth + 1)
    }
  }
  loop(root, 0)

  return result
}
```